Lab_sheet-06

Anish Tiwari

Section-C

Roll no-11

Experiment No. 6

Experiment Name

Implementation and Comparative Analysis of Image Segmentation Techniques using Python and OpenCV

Program Code and Output

Step 1: Import Libraries

In [2]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

In [3]:
```
img = cv2.imread("my_pic01.png")
```

In [8]:
```
from PIL import Image
img=Image.open("my_pic01.png")
img
```

Out[8]:



Step 2: Load Image + Grayscale + Gaussian Blur

In [4]:
```
img = cv2.imread("my_pic01.png")

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

blur = cv2.GaussianBlur(gray, (5, 5), 0)

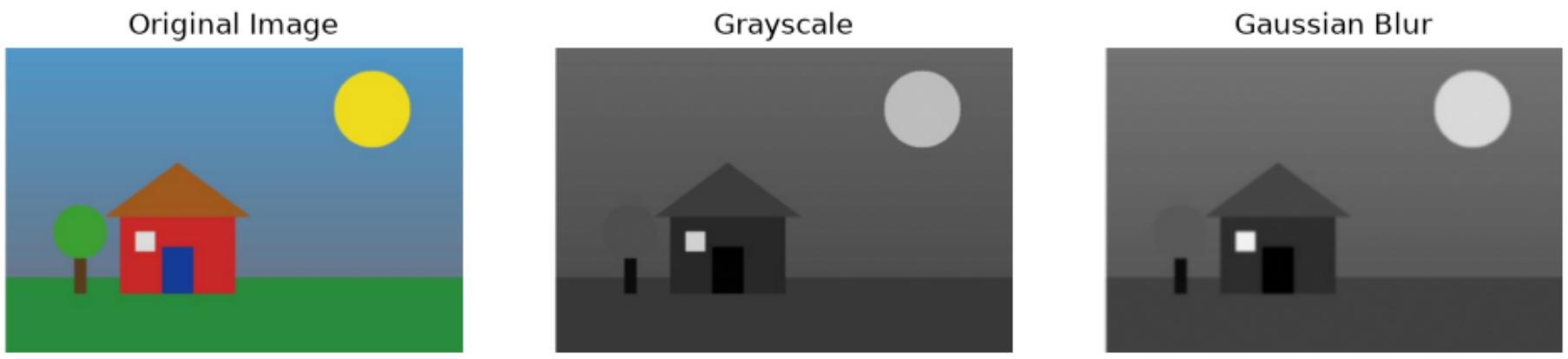
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(blur, cmap="gray")
plt.title("Gaussian Blur")
plt.axis("off")

plt.show()
```



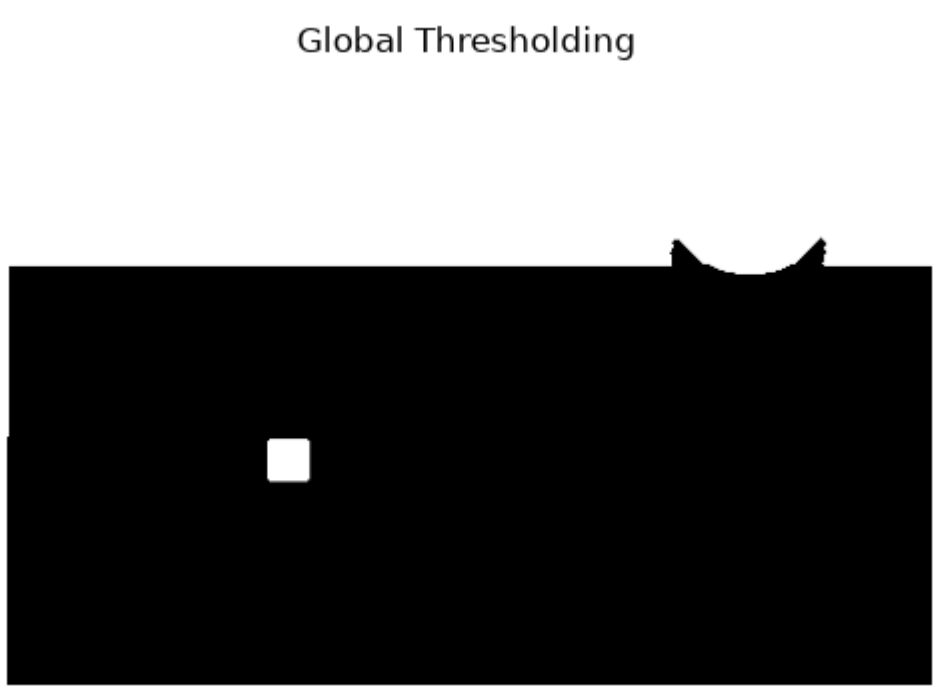
Step 3: Global Thresholding

In [5]:
```
_, global_thresh = cv2.threshold(
    blur,
    127,
    255,
    cv2.THRESH_BINARY
)

plt.figure(figsize=(6, 5))

plt.imshow(global_thresh, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")

plt.show()
```



Step 4: Otsu's Thresholding

In [6]:
```
otsu_value, otsu_thresh = cv2.threshold(
    blur,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

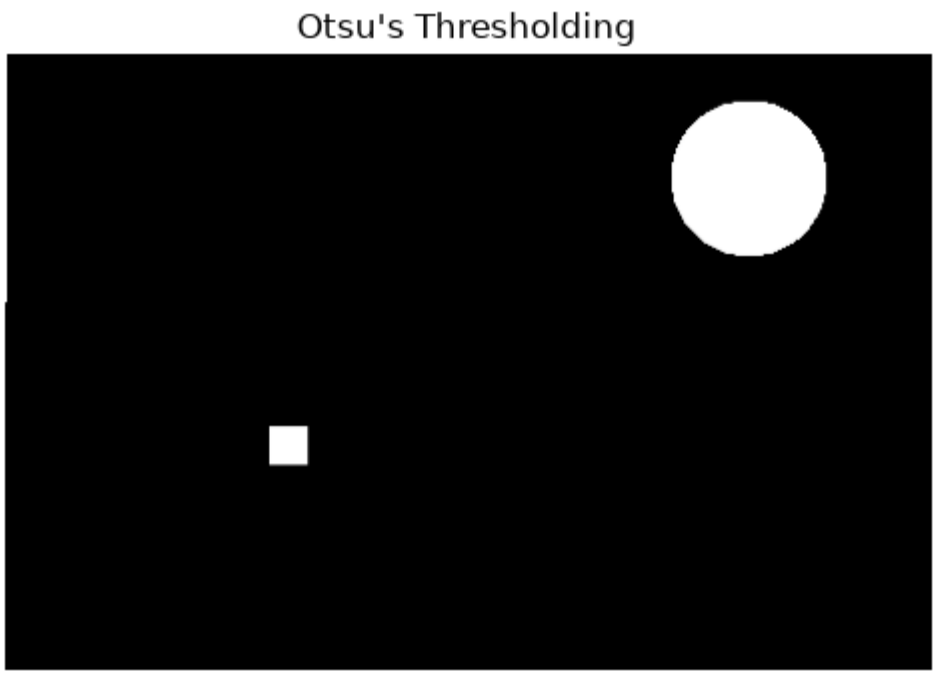
print("Otsu Threshold Value:", otsu_value)

plt.figure(figsize=(6, 5))

plt.imshow(otsu_thresh, cmap="gray")
plt.title("Otsu's Thresholding")
plt.axis("off")

plt.show()
```

```
Otsu Threshold Value: 159.0
```



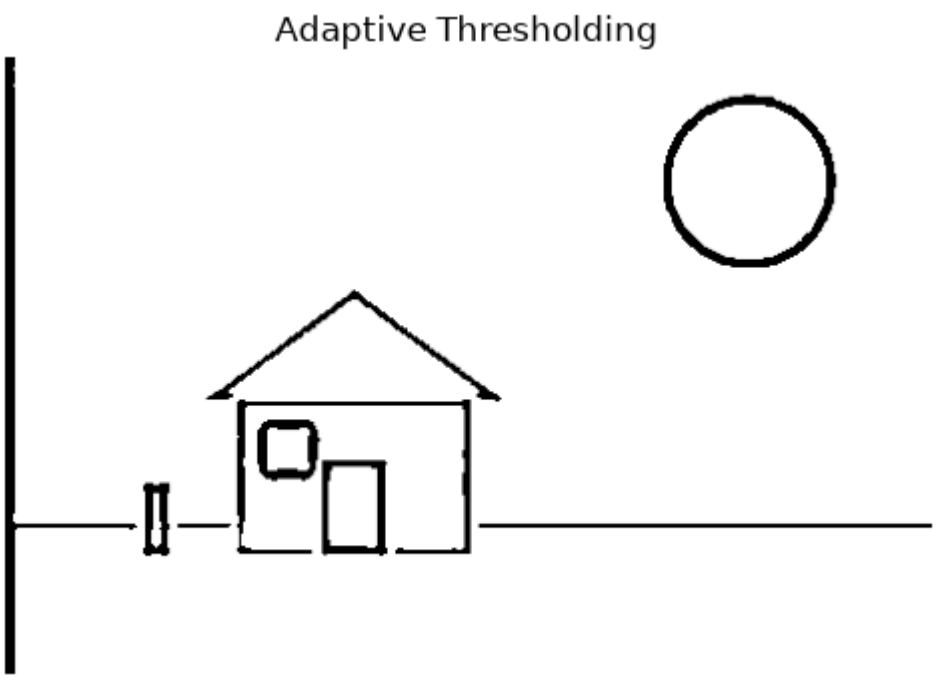
Step 5: Adaptive Thresholding

In [7]:
```
adaptive_thresh = cv2.adaptiveThreshold(
    blur,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

plt.figure(figsize=(6, 5))

plt.imshow(adaptive_thresh, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")

plt.show()
```



Step 6: Watershed Segmentation

In [8]:
```
# Convert threshold image to binary
binary = otsu_thresh

# Remove noise
kernel = np.ones((3, 3), np.uint8)

opening = cv2.morphologyEx(
    binary,
    cv2.MORPH_OPEN,
    kernel,
    iterations=2
)

# Sure background
sure_bg = cv2.dilate(opening, kernel, iterations=3)

# Distance transform
dist_transform = cv2.distanceTransform(
    opening,
    cv2.DIST_L2,
    5
)
```

```
# Sure foreground
_, sure_fg = cv2.threshold(
    dist_transform,
    0.5 * dist_transform.max(),
    255,
    0
)

sure_fg = np.uint8(sure_fg)

# Unknown region
unknown = cv2.subtract(
    sure_bg,
    sure_fg
)

# Marker labelling
_, markers = cv2.connectedComponents(sure_fg)

markers = markers + 1

markers[unknown == 255] = 0

# Apply watershed
watershed_img = img.copy()

markers = cv2.watershed(
    watershed_img,
    markers
)

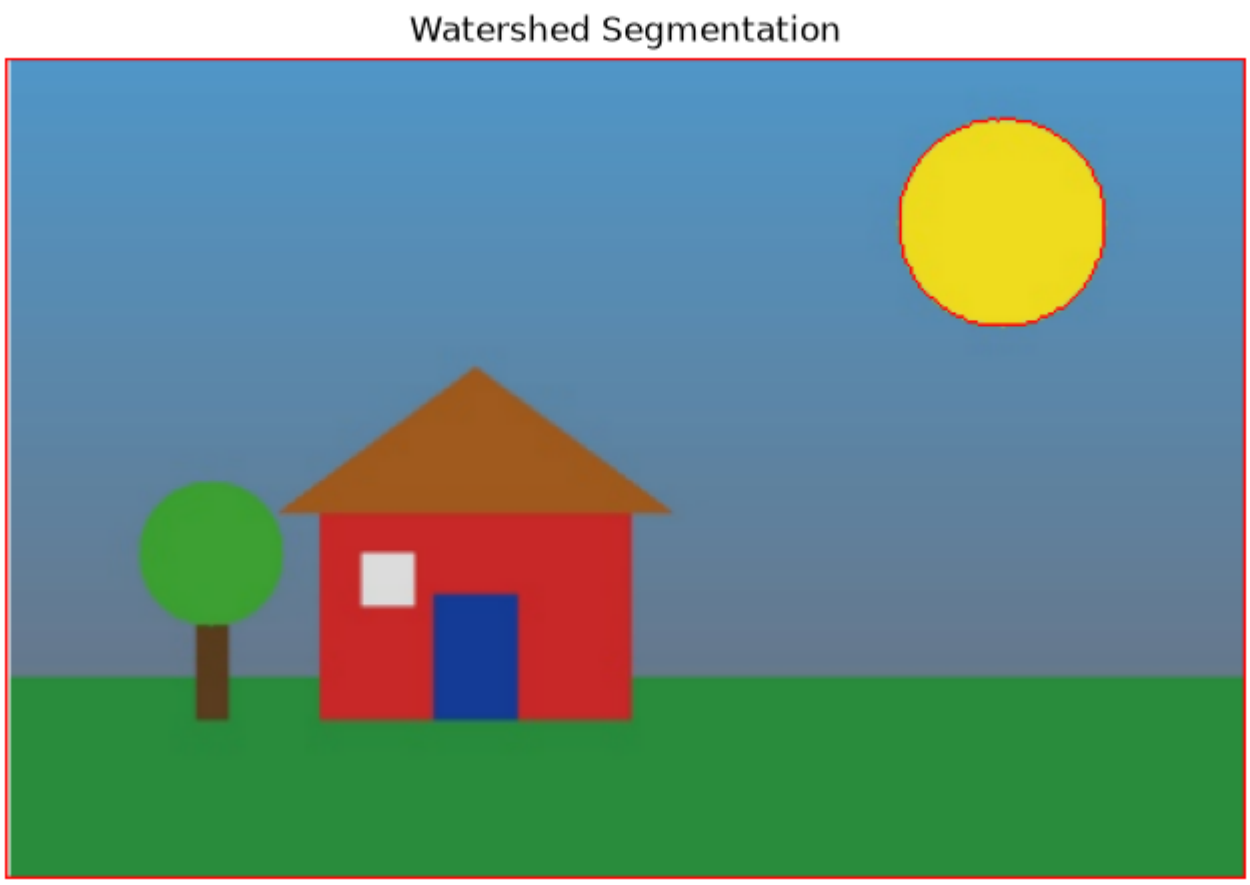
watershed_result = img_rgb.copy()

watershed_result[markers == -1] = [255, 0, 0]

plt.figure(figsize=(8, 6))

plt.imshow(watershed_result)
plt.title("Watershed Segmentation")
plt.axis("off")

plt.show()
```



Step 7: K-Means Color Segmentation

In [9]:
```
# Convert image pixels into a 2D array
pixel_data = img_rgb.reshape(
    (-1, 3)
)

pixel_data = np.float32(pixel_data)

# Number of clusters
k = 3

# Apply K-Means
kmeans = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
```

```
)

labels = kmeans.fit_predict(pixel_data)

# Cluster centers
centers = np.uint8(kmeans.cluster_centers_)

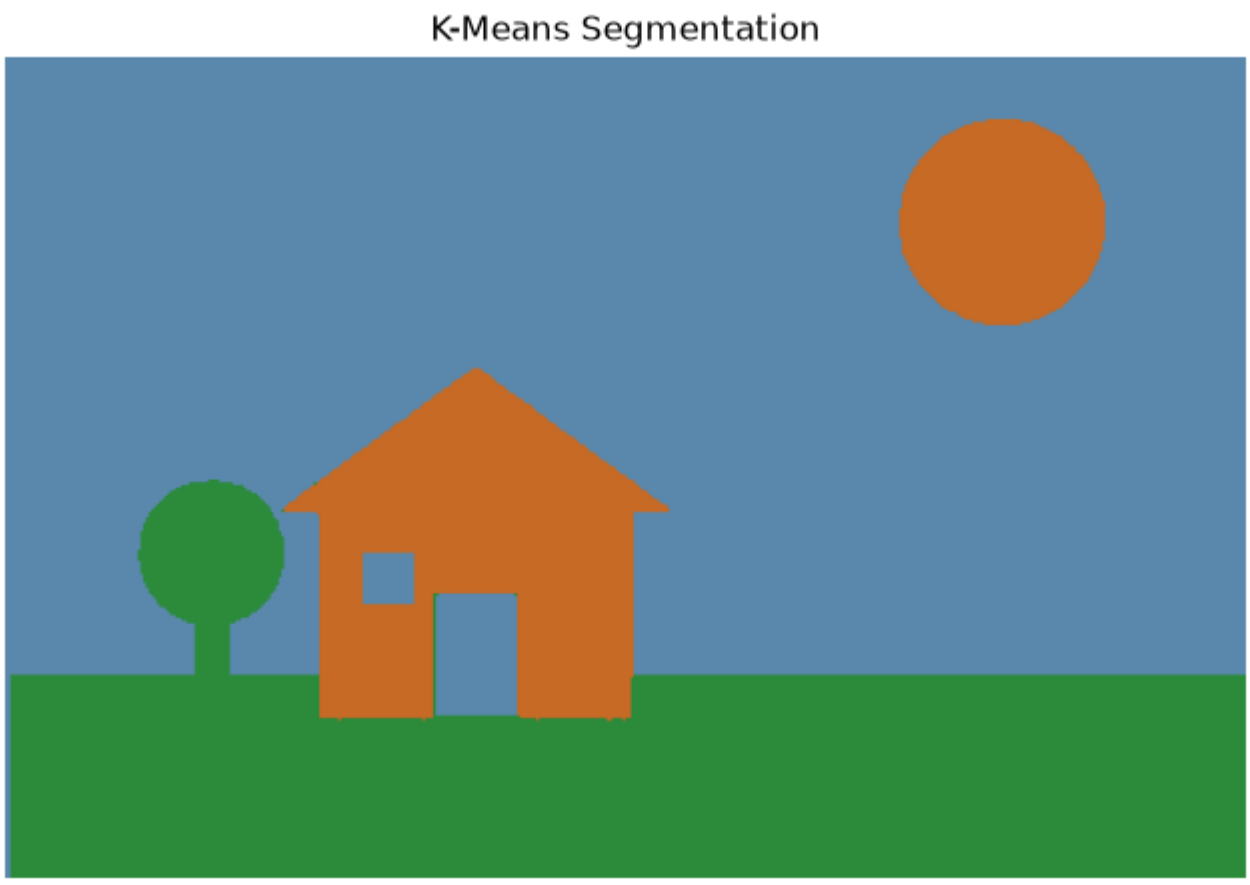
# Replace pixels with cluster centers
segmented = centers[labels]

# Reshape back to image
segmented = segmented.reshape(img_rgb.shape)

plt.figure(figsize=(8, 6))

plt.imshow(segmented)
plt.title("K-Means Segmentation")
plt.axis("off")

plt.show()
```



Step 8: Compare All Techniques

In [10]:
```
plt.figure(figsize=(14, 8))

plt.subplot(2, 3, 1)
plt.imshow(img_rgb)
plt.title("Original")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(global_thresh, cmap="gray")
plt.title("Global Threshold")
plt.axis("off")

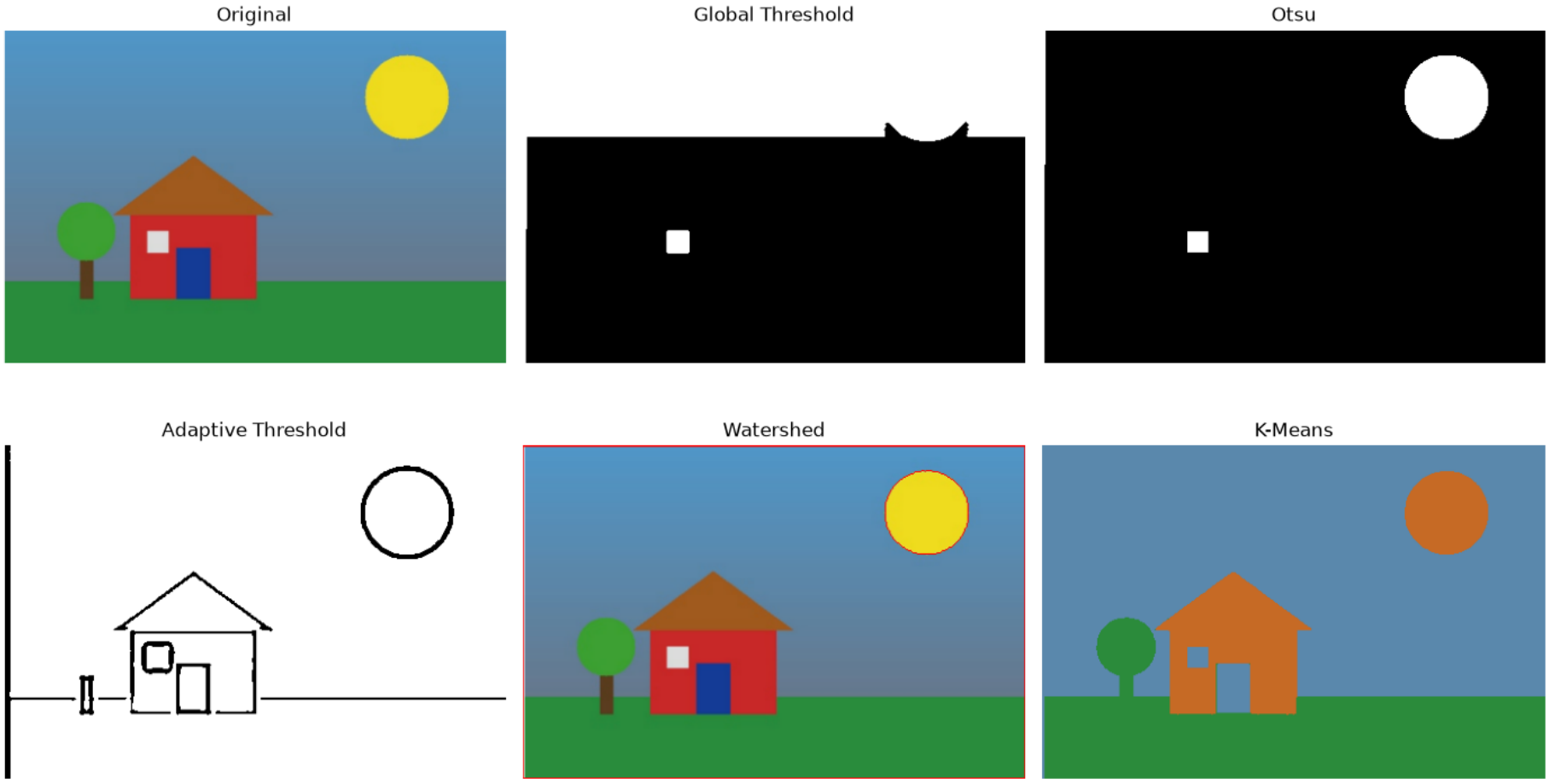
plt.subplot(2, 3, 3)
plt.imshow(otsu_thresh, cmap="gray")
plt.title("Otsu")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(adaptive_thresh, cmap="gray")
plt.title("Adaptive Threshold")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(watershed_result)
plt.title("Watershed")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(segmented)
plt.title("K-Means")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Step 9: Analyze Results

In [11]:
```
print("========== SEGMENTATION ANALYSIS ==========")

print("1. Global Thresholding:")
print("   Simple and fast, but depends on a fixed threshold.")

print("\n2. Otsu's Thresholding:")
print("   Automatically selects a threshold value.")

print("\n3. Adaptive Thresholding:")
print("   Works better with uneven illumination.")

print("\n4. Watershed:")
print("   Useful for separating touching objects.")

print("\n5. K-Means:")
print("   Groups pixels based on color similarity.")
```

```
========== SEGMENTATION ANALYSIS ==========
1. Global Thresholding:
   Simple and fast, but depends on a fixed threshold.

2. Otsu's Thresholding:
   Automatically selects a threshold value.

3. Adaptive Thresholding:
   Works better with uneven illumination.

4. Watershed:
   Useful for separating touching objects.

5. K-Means:
   Groups pixels based on color similarity.
```

Step 10: Final Observation

In [12]:
```
print("""
Conclusion:
Image segmentation divides an image into meaningful regions.
Global, Otsu and Adaptive Thresholding are useful for intensity-based
segmentation. Watershed is useful for separating connected objects,
while K-Means is useful for color-based segmentation.
""")
```

```
Conclusion:
Image segmentation divides an image into meaningful regions.
Global, Otsu and Adaptive Thresholding are useful for intensity-based
segmentation. Watershed is useful for separating connected objects,
while K-Means is useful for color-based segmentation.
```

Viva Questions and Answers

1. What is image segmentation?

   Image segmentation divides an image into meaningful regions or objects.

2. What is the difference between segmentation and classification?

   Segmentation divides an image into regions, while classification assigns a label to an image or object.

3. What is Global Thresholding?

   It uses a single fixed threshold value to divide pixels into different regions.

4. What is Otsu Thresholding?

   Otsu's method automatically selects an optimal threshold from the image histogram.

5. What is Adaptive Thresholding?

   It calculates different threshold values for different regions of an image.

6. Why is Watershed used?

   Watershed is mainly used to separate touching or overlapping objects.

7. What is K-Means segmentation?

   K-Means groups pixels into clusters based on their color or intensity similarity.

8. What are common challenges in image segmentation? Uneven illumination Noise Complex backgrounds Similar object and background colors Unclear object boundaries

9. Where is image segmentation used? Medical image analysis Object detection Autonomous vehicles Industrial inspection Satellite image analysis

10. Which segmentation method is best?

    There is no single best method. The appropriate method depends on the image and application requirements.

In [ ]: